

Neural Networks and Deep Learning

BMED365: Topic List D01–D13

BMED365

Spring 2026

Overview

- 1 Basic Neural Networks
- 2 Training Neural Networks
- 3 Convolutional Neural Networks (CNN)
- 4 Regularization and Advanced Techniques

D01: Compare biological and artificial neurons

Biological neuron:

- **Dendrites:** Receives signals
- **Cell body:** Processes
- **Axon:** Transmits
- **Synapse:** Connection to next neuron
- Complex electrochemical activity

Artificial neuron (perceptron):

- **Inputs** $x_1, x_2, \dots$: Receives data
- **Vekter** $w_1, w_2, \dots$: Synaptic strength
- **Sum:** $z = \sum w_i x_i + b$
- **Aktivering:** $a = f(z)$ (f = activation function)
- Mathematical simplification

Key similarity

Both sum incoming signals og “fires” (activates) hvis total stimulering exceeds a threshold.

Important difference

Artificiale nevroner er **greatly simplified models** – de fanger ikke hjernens fulle kompleksitet.

D02: Describe the structure of a multilayer perceptron (MLP)

MLP architecture:

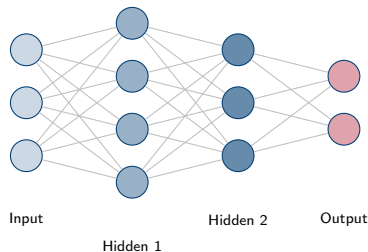
- **Input layer:** Receives features (no processing)
- **Hidden layers:** 1 or flere layers of neurons
- **Output layer:** Produces prediction

Fully connected (dense):

- Each neuron in one layer is connected to **alle** neurons i neste layers
- Many parameters (weights and biases)

“Dyp” læring:

- ≥ 2 hidden layers = deep model
- More layers $\rightarrow$ more abstract representations



D03: Explain what an activation function is (ReLU, sigmoid)

Why activation functioner? Without them: The entire network = linear transformation. Introduserer ikke-linearitet.

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$

- Output: $(0, 1)$
- Problem: *vanishing gradients*
- Used in output for binary classification

ReLU: $\text{ReLU}(z) = \max(0, z)$

- Output: $[0, \infty)$
- Modern standard for hidden layers
- Fast, avoids vanishing gradients

Softmax (multi-klasse output)

$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ – gives probability distribution over classes (sums to 1)

D04: Understand the concept of forward propagation

Forward propagation = forward computation

- ❶ **Input:** Data x (features) legges inn i inputlayerset
- ❷ **For each layer:**
 - Compute weighted sum: $z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)}$
 - Apply activation: $a^{(l)} = f(z^{(l)})$
- ❸ **Output:** Prediksjon $\hat{y}$ fra siste layers



Key point

Forward propagation calculates prediction – no learning happens here. Learning happens through backpropagation og update av vekter.

D05: Explain backpropagation at a conceptual level

Backpropagation = backward computation of error

Main idea:

- 1 **Beregn feil:** Sammenlign prediction $\hat{y}$ med fasit y
- 2 **Propagate backward:** How much did each weight contribute to the error?
- 3 **Chain rule:** Differentiate error with respect to hver vekt i nettverket
- 4 **Oppdater vekter:** Adjust to reduce error

Intuisjon

Imagine adjusting a complex machine: Backpropagation tells you which screws (weights) du shall turn, og in which direction, to improve the result.

Matematisk (forenklet)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

(Chain rule for derivasjon propagerer gradienter bakover gjennom nettverket)

D06: Understand gradient descent and learning rate

Gradient descent:

- Optimization algorithm to **minimize the loss function**
- Følger den **negative gradienten** (steepest descent)

Update rule:

$$w_{ny} = w_{\text{gammel}} - \eta \cdot \frac{\partial L}{\partial w}$$

Learning rate η :

- Bestemmer **steglengde**
- Too large: Jumps over minimum
- Too small: Slow convergence
- Typisk: 0.001–0.01

Variants:

- **Batch GD**: All data per update
- **Stochastic GD**: One example at a time
- **Mini-batch GD**: Small batches (32–128)
- **Adam**: Adaptive learning rate (popular!)

Epoch

En **epoch** = én gjennomgang av hele training datasetet

D07: Know about loss functions (cross-entropy, MSE)

Loss function (loss function):

- Måler **hvor feil** the model's predictions are
- The goal: Minimize loss during training

MSE (Mean Squared Error):

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Brukes for **regression**
- Penalizes large errors heavily
- Continuous output

Cross-Entropy:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

- Brukes for **classification**
- Binary: $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
- Measures divergence between probability distributions

Choosing the right loss

Regression: MSE/MAE **Binær:** Binary CE **Multi-klasse:** Categorical CE

D08: Describe a convolutional neural network (CNN)

CNN = specialized for image data

- Utnytter **romlig struktur** i bilder
- Far fewer parameters than fully connected network
- Translation invariant – recognizes patterns regardless of position

Typical CNN architecture:



Hierarchical learning:

- Early layers: Simple features (edges, textures)
- Later layers: Complex features (shapes, object parts)
- Final layer: High-level concepts (objects, categories)

D09: Explain what a convolution filter does

Convolution filter (kernel):

- Small matrix (e.g. 3×3) som **slides over the image**
- Calculates dot product between filter and image patch
- Produserer et **feature map**

Example: Edge detection

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- Detects horizontal edges
- Different filters $\rightarrow$ different features
- CNN **lærer** optimale filtre!

Key concepts:

- **Stride:** How far the filter moves
- **Padding:** Adding pixels at the border
- **Channel:** RGB = 3 kanaler
- **Feature map:** Output from convolution

Weight sharing

Same filter used across entire image $\rightarrow$ dramatically fewer parameters enn fullt koblet nettverk.

D10: Describe pooling layers and their function

Pooling = downsampling of feature maps

Most common type: Max pooling

- Velger **maksvalue** in each region (e.g. 2×2)
- Reduces spatial size by factor of 2

Advantages of pooling:

- **Reduces computation** – fewer parameters
- **Translation invariance** – slight shift does not affect output
- **Prevents overfitting** (overfitting)

Example (2×2 max pool):

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \rightarrow 4$$

$$\begin{bmatrix} 5 & 1 \\ 0 & 2 \end{bmatrix} \rightarrow 5$$

Other pooling types

- **Average:** Average
- **Global avg:** Feature map $\rightarrow$ én value

D11: Know about batch normalization and dropout

Batch Normalization:

- Normalizes activations in each mini-batch
- $\hat{x} = \frac{x - \mu}{\sigma}$, then scale/shift
- Advantages:
 - Faster training
 - Stabilizes learning
 - Allows higher learning rate
- Typically placed after convolution, before activation

Dropout:

- “Slår av” tilfældige neuroner under training
- Typisk dropout rate: 0.2–0.5
- Advantages:
 - Reducerer **overfitting**
 - Forces network to be robust
 - Acts as ensemble
- Used only during training!

Regularization techniques

Både batch normalization og dropout help prevent overfitting and improve generalization.

D12: Understand the concept of transfer learning

Transfer learning:

- Reuse a model trained on one problem for a new problem
- Spesielt nyttig når du har **lite data**

Typical approach:

- 1 Take a pretrained model (e.g. trained on ImageNet – millions of images)
- 2 Freeze early layers (general features)
- 3 Replace/train final layer for your specific task
- 4 (Valgfritt) Fine-tune entire model with low learning rate

Why does it work?

Early layers lærer **general features** (edges, textures) that are useful for many tasks. Only the final layers are task-specific.

Example in medicine

Train on millions of natural images → Fine-tune on thousands of X-ray images → Better results than training from scratch!

D13: Know about advanced architectures (ResNet, ViT)

ResNet (Residual Networks):

- Introduces **skip connections**
- Allows gradients to flow directly
- Enables very deep networks (50–152+ layers)
- $y = F(x) + x$ (residual block)

Important contribution:

- Solved the problem of degradation in deep networks
- Standard for medical image analysis

ViT (Vision Transformer):

- Applies transformer architecture to images
- Divides image into patches (“tokens”)
- Uses self-attention
- Scales well with data and compute

Modern trend:

- Outperforms CNN on large datasets
- Used in state-of-the-art models

Other important architectures

U-Net: Segmentation (medical classic) | **EfficientNet:** Balanced scaling | **DenseNet:** Dense connections

Summary: Deep Learning

Fundamental:

- D01–D03: Nevroner, MLP, activation functioner
- D04–D07: Forward/backprop, gradient descent, loss functions

CNN:

- D08–D10: Arkitektur, konvolusjonsfiltre, pooling
- Hierarchical learning av bildfeatures

Avansert:

- D11: Batch normalization, dropout (regularisering)
- D12: Transfer learning (gjenbruk av pretrent kunnskap)
- D13: ResNet (dype nettverk), ViT (transformers for bilder)

Lab 2 – Praktisk erfaring

Bygg, tren og evaluate CNN med PyTorch. Bruk Grad-CAM to understand modellbeslutninger.